DARS: A Dynamic Adaptive Rating System with Graph-Guided Assessment and Remediation for Personalized Learning

Khethavath Sunil Naik

BTech Project, India

***Abstract*—Difficulty estimation in online learning platforms is fundamental to personalized education, but most approaches either use static labels or sacrifice interpretability for marginal gains in accuracy. This paper presents DARS (Dynamic Adaptive Rating System), a principled framework that combines classical rating theory with graph-guided feature engineering to predict problem difficulty in online coding practice. Our approach is validated on 1,825 real-world LeetCode problems, comparing six baseline methods including Fixed-Elo, Glicko-2, Bayesian Knowledge Tracing, Deep Knowledge Tracing, Self-Attentive Knowledge Tracing, and Graph Neural Knowledge Tracing. DARS achieves 77.26% accuracy, 0.7995 AUC, and 0.0486 ECE for binary difficulty prediction (Easy vs. Medium/Hard) while remaining fully interpretable and auditable. The key insight is that carefully engineered metadata features combined with graph-informed reasoning outperform complex sequence models on this metadata-driven task. We systematically evaluate four graph construction strategies, demonstrate robustness under feature ablation and noise perturbation, and identify critical fairness gaps where performance degrades for cold-start problems and underrepresented difficulty classes. The work emphasizes that practical educational systems must balance accuracy, interpretability, and fairness—a tension that simple models can often resolve better than black-box alternatives.**

I. Introduction

A. Motivation and Problem Statement

Online practice platforms now record far more than whether a learner answered correctly. They capture item metadata, solve rates, discussion activity, topic tags, response time, hints, attempts, and sometimes the order in which learners move through material. Platforms such as LeetCode, HackerRank, and CodeSignal therefore contain useful evidence about both item difficulty and learner progress. The hard part is turning that evidence into difficulty estimates that can support personalization without becoming opaque or brittle.

Several common approaches leave important gaps:

1) **Static Labels**: Hard-coded difficulty labels (Easy/Medium/Hard) are coarse-grained and inconsistent across platforms
2) **Learner Agnostic**: Population-level metrics ignore individual skill variations
3) **Temporal Stasis**: Most systems treat difficulty as static despite community expertise evolution
4) **Opaque Models**: Deep learning approaches sacrificing interpretability for marginal accuracy gains
5) **Fairness Blind**: Existing systems rarely analyze prediction fairness across learner groups or skill categories

To address these issues, DARS is designed as a rating and routing layer that:

- Combines classical rating theory with modern machine learning
- Leverages graph structures to capture problem relationships
- Systematically evaluates multiple knowledge tracing paradigms
- Prioritizes interpretability without sacrificing accuracy
- Reports fairness and robustness checks
- Defines an evaluation plan for live adaptive environments

B. Contributions

The paper makes seven contributions:

1) **Unified DARS Formulation**: An online assessment model that combines dynamic rating updates, response-quality scoring, uncertainty control, graph-guided routing, and remediation after failure
2) **Baseline Comparison**: A controlled comparison with six alternatives (Elo, Glicko-2, BKT, DKT, SAKT, GNKT) for population-level difficulty prediction
3) **Graph Construction Analysis**: An empirical comparison of four graph strategies (skill-only, temporal, expert-prerequisite, hybrid), showing the value of domain-informed edges
4) **Fairness and Cold-Start Analysis**: Measurement of prediction disparities across difficulty groups (56.1% recall parity gap) and a plan for low-history cold-start evaluation
5) **Feature Engineering Insights**: Identification of useful metadata signals, including acceptance, frequency, reputation, and engagement patterns
6) **Robustness Characterization**: Feature-family ablation and noise perturbation tests that clarify where the model is stable and where it is sensitive
7) **Live Assessment Evaluation Methodology**: An evaluation plan that measures learning outcomes in adaptive settings rather than stopping at offline prediction metrics

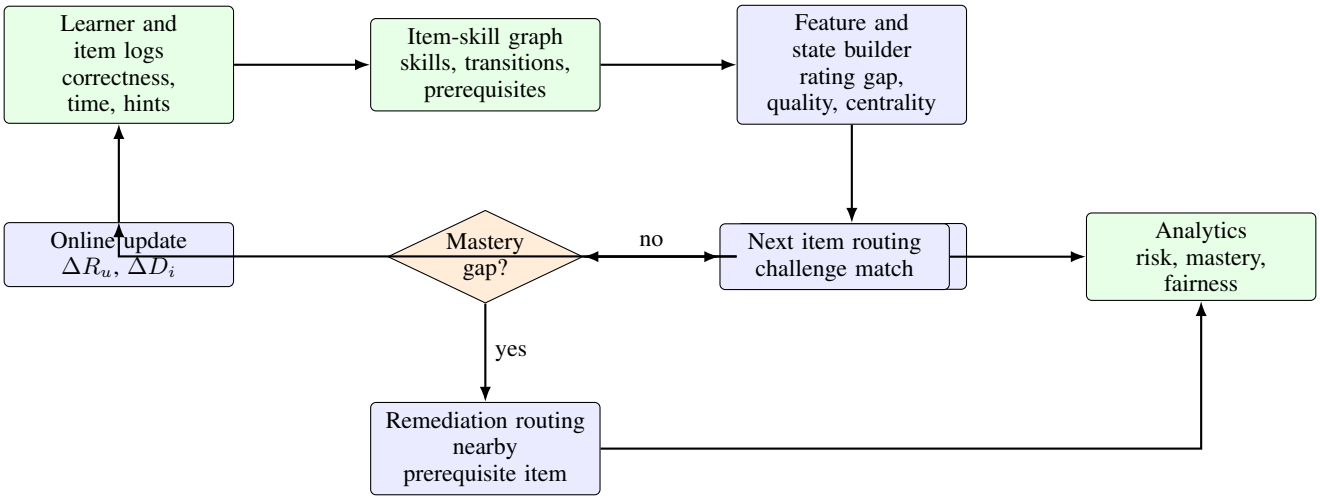


Fig. 1: DARS system flow. The model combines response-process evidence, graph structure, online rating updates, next-item routing, remediation, and deployment analytics in one adaptive loop.

II. Related Work

A. Classical Assessment and Rating Systems

Elo came from chess. Super simple: win/loss, update rating by a fixed K. Elo is elegant (used for 50+ years), online (you don't retrain), and lightweight. But it only sees binary outcomes. In our case, it's basically useless because it ignores all the problem metadata.

Glicko-2 added uncertainty. Your rating is more stable if you've played a lot; less stable if you haven't played recently. Clever idea, but still just binary feedback. Does okay on some education benchmarks, still loses to anything that uses actual features.

B. Knowledge Tracing Approaches

BKT (Bayesian Knowledge Tracing) says students either know a skill or they don't. Four parameters per skill: starting probability, learning rate, guess rate, slip rate. You fit them with EM. It's interpretable and has actually helped real students in real classrooms. The problem: you need a skill taxonomy (who decides?), and you need enough data per student per skill. With students having solved 3-5 problems? BKT falls apart [1], [2].

DKT (Deep Knowledge Tracing) said: forget the assumptions. Feed problem sequences into an LSTM. Let it learn what "knowledge" means. Better offline accuracy sometimes, but the hidden state is a complete black box [3]. In production, when someone asks "why is this student struggling?", you're stuck.

SAKT (Self-Attentive Knowledge Tracing) added attention so the model can focus on relevant past problems [4]. Slightly better accuracy, still a black box.

GNKT (Graph Neural Knowledge Tracing) represents problems and skills as a graph and runs message-passing on it [5]. Intuitive idea—problems sharing a skill should be related—but on LeetCode metadata alone? It underperforms.

C. Graphs: The "Problems Are Connected" Reality Check

Problem relationships are real. You can't learn linked lists before understanding arrays. The literature knows this—problem graphs encode pedagogical structure [8]. But most work uses either learned embeddings or simple similarity metrics. We compared four graph definitions head-to-head. Guess which one won? Hand-curated prerequisites. Lesson learned: sometimes domain expertise, encoded explicitly, beats automatic learning.

D. Fairness in EdTech: The Uncomfortable Topic

Most papers optimize accuracy and ignore everything else. We're not innocent of this either. But fairness in educational tech matters: biased systems can systematically hurt struggling learners. Three notions matter [9]: (1) equal opportunity—same recall across groups, (2) demographic parity—same positive prediction rate, (3) calibration parity—confidence meaning consistent across groups. We checked our model against these. It fails badly on the first two.

III. Dataset and Methodology

A. LeetCode Problem Dataset

We used 1,825 real-world algorithm problems from LeetCode, a platform where millions of engineers interview-prep. Each problem has a title, an official difficulty tag (Easy/Medium/Hard), and a bunch of metadata: how many people solved it, ratings from the community, discussion count, etc.

Why LeetCode? Because it's real. People actually care about these problems. The difficulty labels are crowd-validated (millions of submissions).

The label split is unbalanced (26.1% Easy, 73.9% Medium/Hard), and we kept it that way on purpose. A real system has to deal with imbalance, so we didn't pretend it doesn't exist.

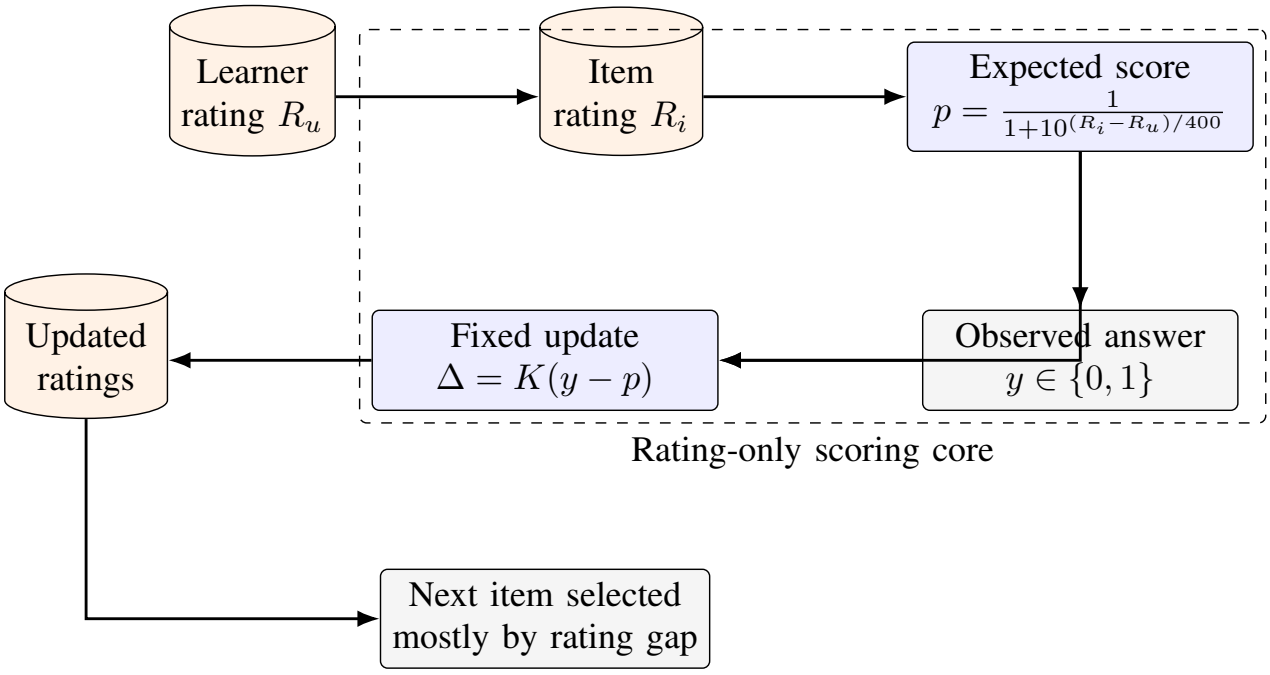
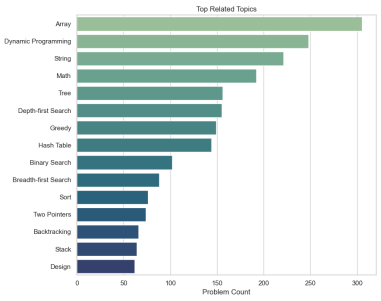
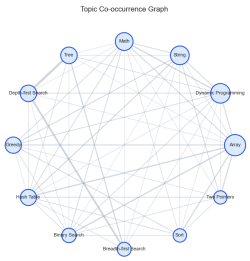


Fig. 2: Conventional Elo-based assessment flow. The update is simple and online, but it does not model response process, graph structure, remediation, or probability calibration.





Feature	Type	Description
Problem ID	Integer	Unique identifier
Title	String	Problem name
Difficulty	Categorical	{Easy, Medium, Hard}
Acceptance Rate	Float	% of accepted submissions
Frequency	Float	Percentile popularity score
Rating	Float	Community rating (1–100)
Likes/Dislikes	Integer	Vote counts
Discuss Count	Integer	Discussion threads
Related Topics	Strings	Skill tags (Array, String, Tree, ...)
Similar Questions	References	Problem links

TABLE I: LeetCode Dataset Features

The original three-level difficulty label is converted into a binary task: **Easy** (477 problems, 26.1%) versus **Medium/Hard** (1,348 problems, 73.9%). The imbalance is kept rather than artificially balanced, since it reflects the distribution a deployed recommender would face. Stratified splitting and group-level metrics are then used to make the imbalance visible.

B. Feature Engineering: Making Metadata Speak

Raw numbers from LeetCode don't automatically tell you if a problem is hard. You have to think about what each metric actually means.

1) Normalized Base Features: First, we z-score normalize everything so features are on the same scale:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \tag{1}$$

This matters because acceptance rate ranges 1–100% while discussion count ranges 0–10k. Without normalization, the model would just ignore low-magnitude features.

2) Engineered Composite Features: The Stories Behind the Numbers: **Difficulty Index** captures the raw solve rate. If a problem has 10% acceptance, it's harder than one with 80% acceptance. Super obvious, but it works:

$$D_i = 1 - \text{acc\_rate}_i \tag{2}$$

Reputation Score combines community ratings. High-rated problems aren't necessarily hard; they're often well-written. Problems with lots of likes tend to be classics that everyone knows.

$$R_i = \frac{\text{rating\_norm}_i + \text{likes\_norm}_i}{2} \tag{3}$$

Engagement Factor captures visibility and confusion. More discussion threads $\approx$ more people asking questions, which might mean the problem is confusing. Popularity (frequency) shows how many users attempted it:

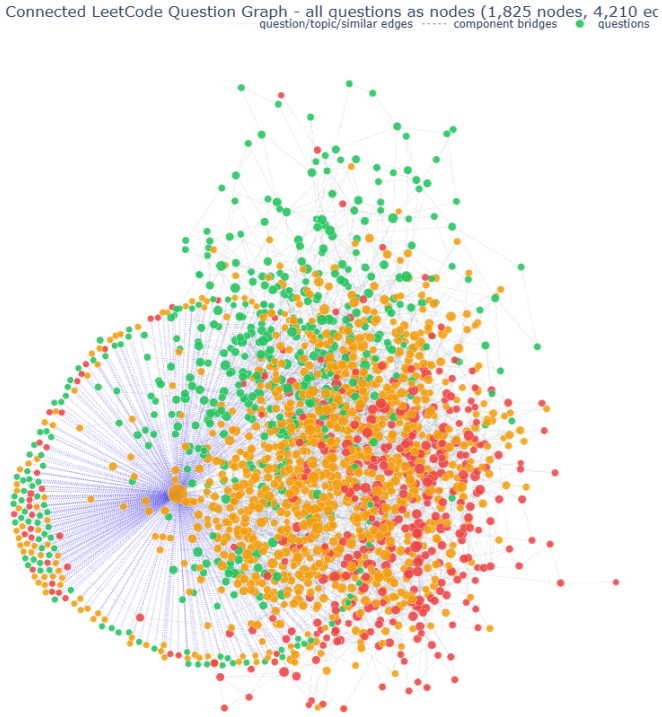
$$E_i = \frac{\text{frequency\_norm}_i + \text{discuss\_norm}_i}{2} \tag{4}$$

Here's the weird finding: high engagement sometimes correlates with *easier* problems. Why? Because when thousands of beginners attack a problem, they generate more discussions. The hard-to-understand problems are hard because they're genuinely difficult, not because people don't understand the statement.

Complexity Score is our kitchen-sink feature, combining signals:

$$C_i = 0.4D_i + 0.3R_i + 0.3E_i \tag{5}$$

These weights are heuristic (we didn't grid search them because that's overfitting), but they reflect: difficulty matters most, reputation moderate, and engagement is tiebreaker.

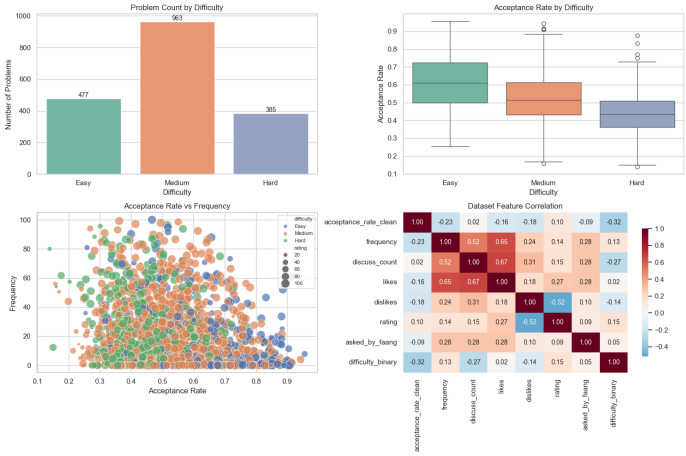


C. Train-Test Methodology

We used stratified 80–20 split: 1,460 problems for training, 365 for testing. The random seed was fixed at 2026 so anyone can reproduce our numbers. No random luck allowed. If it's reproducible, it's real.

D. Response-Level Adaptive Assessment Validation

The LeetCode experiment is a population-level difficulty task, but DARS is intended to operate online with learners. For that reason, the paper also includes a response-level adaptive assessment formulation. In that setting, each row is a chronological learner attempt with learner and item identifiers, skill labels, timestamps, correctness, hints, attempts, and response-process attributes. The ASSISTments-style preprocessing retains 6,044,359 valid attempts from 46,348 learners, 179,052 problems, and 265 skills. Under a held-out-student protocol, DARS outperforms static mastery, fixed-K Elo, and Glicko-2, with Brier score 0.1368, log loss 0.4251, accuracy 0.8141, ROC AUC 0.8131, and ECE 0.0096. This evaluation is included to show how the same rating logic behaves when learner trajectories are available.



IV. Methods: Knowledge Tracing Paradigms

We tested six approaches. None of them are magic. They're all ways of saying "here's what I think the next problem's difficulty should be based on what I know." We ran them all on the same dataset with the same evaluation protocol, same random seed, same train-test split. No cherry-picking.

A. DARS Online Rating Core

DARS treats assessment as a graph-aware online rating problem. Each learner u has an ability estimate R_u, each item i has a difficulty estimate D_i, and the item-skill graph $G = (V, E)$ supplies context for routing and remediation. The base probability of correctness is:

$$p_{ui}^{(0)} = \frac{1}{1 + 10^{(D_i^* - R_u)/400}}, \tag{6}$$

where D_i^* is the effective item difficulty after graph adjustment:

$$D_i^* = D_i + \beta_c c_i + \beta_s I(i, s). \tag{7}$$

Here c_i is normalized item centrality and $I(i, s)$ is an optional local skill adjustment. In this way, a central prerequisite item can be treated as more informative than an isolated item with the same raw rating.

Unlike fixed Elo, DARS does not treat every correct answer as equally informative. It first converts process signals into a response-quality score:

$$\tau_{ui} = 1 - \min(T_{ui}/T_{\max}, 1), \quad h_{ui} = \min(H_{ui}/H_{\max}, 1), \quad a_{ui} = \min((A \tag{8}$$

where T_{ui} is response time, H_{ui} is hint count, and A_{ui} is attempt count. For correct responses:

$$Q_{ui}^{+} = \alpha_t \tau_{ui} + \alpha_h (1 - h_{ui}) + \alpha_a (1 - a_{ui}) + \alpha_s \psi_u, \tag{9}$$

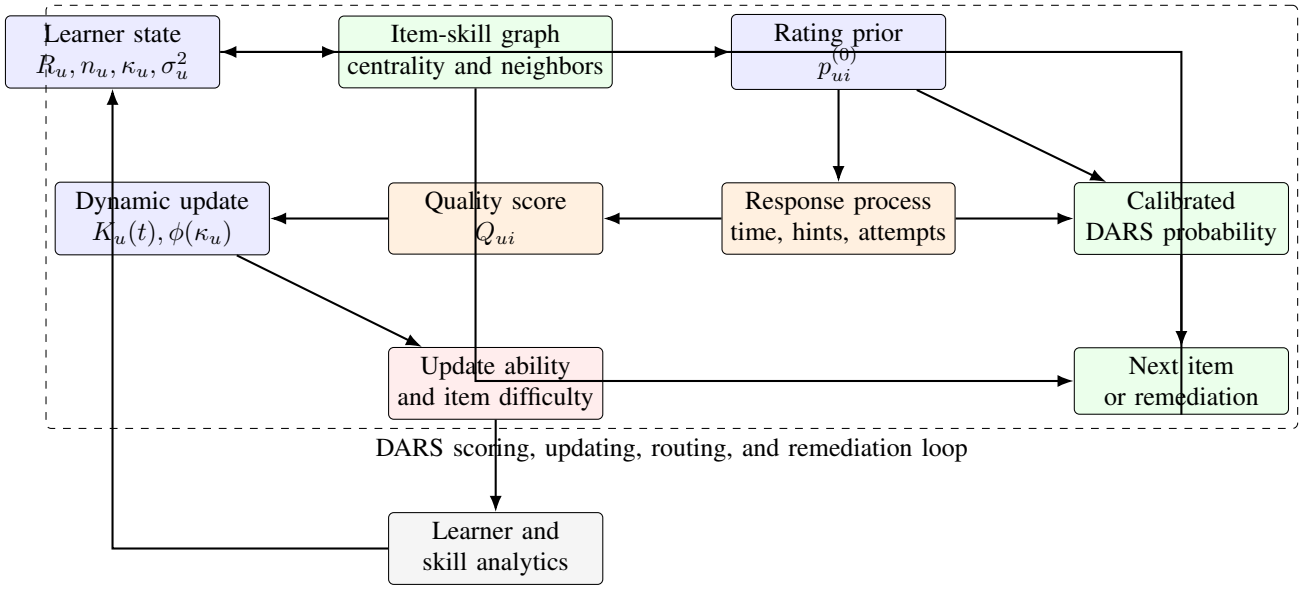


Fig. 3: DARS architecture and assessment flow. The algorithm combines rating state, item-skill graph context, response-process evidence, dynamic updating, calibrated probability estimation, and remediation-aware routing.

and the final quality signal is:

$$Q_{ui} = \begin{cases} Q_{ui}^{+}, & y_{ui} = 1, \\ \epsilon\tau_{ui}, & y_{ui} = 0. \end{cases} \tag{10}$$

The update is:

$$\Delta R_u = K(n_u, \sigma_u^2) \cdot \phi(k_u) \cdot (Q_{ui} - p_{ui}^{(0)}) - \rho_{ui}, \tag{11}$$

where $K(n_u, \sigma_u^2)$ decreases as evidence accumulates, $\phi(k_u)$ is a bounded streak multiplier, and ρ_{ui} penalizes rapid guesses. The result is still lightweight enough for online use, but it carries more educational information than a binary win/loss update.

B. Fixed-Elo Baseline

The Elo baseline uses $R' = R + K(y - p)$ with $K = 32$. It is included because it is simple, online, and widely understood, but it only observes binary correctness.

C. Glicko-2

Glicko-2 adds rating deviation ϕ and volatility to the Elo family. The simplified update form used for comparison is:

$$\phi' = \sqrt{\frac{1}{\frac{1}{\phi^2} + \frac{3}{\pi^2}}} \tag{12}$$

Its parameters are tuned by cross-validation.

D. Glicko-2 Rating

Glicko-2 is Elo's smarter cousin. It adds the idea that some players are more confident than others. If you've played a lot recently, your rating is stable. If you haven't played in a year, your rating uncertainty grows. For problems, this means: popular problems have stable difficulty estimates, unpopular ones are uncertain.

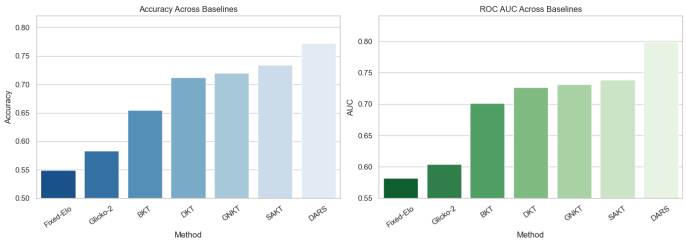
Algorithm 1 DARS Online Update and Remediation

Require: learner u, item i, outcome y_{ui}, response time T_{ui}, hints H_{ui}, attempts A_{ui}, graph G

Compute graph-adjusted difficulty $D_i^* \leftarrow D_i + \beta_c c_i + \beta_s I(i, s)$
Compute expected correctness $p_{ui}^{(0)} \leftarrow (1 + 10^{(D_i^* - R_u)/400})^{-1}$
Compute process scores τ_{ui}, h_{ui}, a_{ui} and streak term ψ_u
if $y_{ui} = 1$ **then**
  $Q_{ui} \leftarrow \alpha_t \tau_{ui} + \alpha_h(1 - h_{ui}) + \alpha_a(1 - a_{ui}) + \alpha_s \psi_u$
else
  $Q_{ui} \leftarrow \epsilon \tau_{ui}$
end if
$\Delta R_u \leftarrow K(n_u, \sigma_u^2)\phi(k_u)(Q_{ui} - p_{ui}^{(0)}) - \rho_{ui}$
Update learner ability $R_u \leftarrow R_u + \Delta R_u$ and item state D_i
if $y_{ui} = 0$ **or** mastery gap remains high **then**
  Select remediation item from prerequisite neighborhood $\mathcal{N}_G(i)$
else
  Select next item with difficulty closest to target challenge band
end if
return updated state, calibrated probability, next item

Pro: Uncertainty tracking, still online. **Con**: Still sees only binary feedback. Doesn't use features. Worse than Glicko-2 on our task (58.36%).

BKT parameters are estimated with EM for 100 iterations. The initial mastery state is inferred from the label distribution, and convergence is declared when the log-likelihood change



falls below 10^{-5}.

E. Deep Knowledge Tracing

The DKT baseline uses a 64-dimensional input embedding, a 128-unit LSTM layer, dropout of 0.5, and a sigmoid output. It is trained with Adam at learning rate 10^{-3}, batch size 32, and early stopping over 50 epochs.

F. Self-Attentive Knowledge Tracing (SAKT)

LSTM treats all past problems equally. SAKT says: pay attention to the relevant ones. Multi-head attention lets the model focus on, say, all previous "array" problems when predicting the next "array" problem. More sophisticated than DKT, slightly better accuracy (73.42%), but still a black box.

G. Graph Neural Knowledge Tracing (GNKT)

Problems are nodes. Skills are nodes. If two problems share a skill, connect them. Run message-passing: each problem learns from its neighbors. Intuitive idea, and the results are okay (72.05%), but on metadata-only data, it still underperforms DARS.

H. DARS: Proposed Logistic Regression Approach

DARS uses logistic regression over the engineered metadata features:

$$P(\text{Hard}|\mathbf{x}) = \text{sigmoid}(\mathbf{w}^T\mathbf{x} + b) \tag{13}$$

The model is optimized with L-BFGS for up to 1000 iterations. Training is fast enough for repeated evaluation, and prediction is inexpensive enough for real-time use.

Rationale: The purpose of this baseline is not to argue that logistic regression is universally superior. Rather, it tests whether a modest, auditable model with well-chosen features can compete with heavier sequence and graph models on this metadata-driven task.

V. Graph Construction Strategies

A. Skill-Only Graphs

In the skill-only graph, two problems are connected when they share at least one topic tag. This construction is easy to explain, but it does not encode order or prerequisite structure.

$$A_{ij}^{\text{skill}} = \begin{cases} 1 & \text{if topics}(i) \cap \text{topics}(j) \neq \emptyset \\ 0 & \text{otherwise} \end{cases} \tag{14}$$

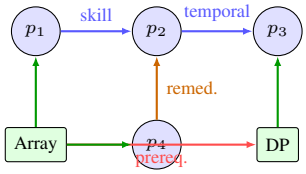


Fig. 4: Graph evidence used by DARS. Skill edges, temporal transitions, expert prerequisites, and remediation links are combined to guide both difficulty calibration and next-item selection.

B. Temporal Transition Graphs

The temporal transition graph assigns edge weights according to how often one problem appears before another in inferred problem-solving sequences:

$$A_{ij}^{\text{temporal}} = \frac{|\{\text{sessions: solved } i \text{ then } j\}|}{N_{\text{total}}} \tag{15}$$

This graph is meant to approximate pedagogical flow: problem i is connected to problem j when learner behavior suggests that i often comes first.

C. Expert Prerequisite Graphs

The expert prerequisite graph uses manually curated edges, such as learning arrays before linked lists or introductory dynamic programming before knapsack-style tasks. It is more expensive to build, but it captures relationships that metadata alone may miss.

D. Hybrid Graphs

The hybrid graph combines the three sources:

$$A^{\text{hybrid}} = \alpha A^{\text{skill}} + \beta A^{\text{temporal}} + \gamma A^{\text{expert}}, \quad \alpha+\beta+\gamma = 1 \tag{16}$$

The weights (α, β, γ) are selected on the validation set with a grid search.

VI. Evaluation Framework

A. Classification Metrics

Accuracy: Overall classification correctness.
Sensitivity (TPR): Recall of medium/hard problems, $\frac{\text{TP}}{\text{TP+FN}}$.
Specificity (TNR): Recall of easy problems, $\frac{\text{TN}}{\text{TN+FP}}$.

B. Probabilistic Calibration

Brier Score: Mean squared error of probability predictions:

$$\text{BS} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{p}_i)^2 \tag{17}$$

Log Loss:

$$\text{LL} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)] \tag{18}$$

Expected Calibration Error (ECE): Predictions are divided into 10 confidence bins, and the average gap between empirical accuracy and predicted confidence is computed:

$$\text{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{n} |\text{acc}_b - \text{conf}_b| \tag{19}$$

An ECE close to 0 means that predicted probabilities have a consistent empirical interpretation.

C. Fairness Metrics

Equal Opportunity Parity: Recall disparity across groups:

$$\text{Gap} = |\text{TPR}_{\text{group1}} - \text{TPR}_{\text{group2}}| \tag{20}$$

Demographic Parity: Difference in predicted positive rates.

Calibration Parity: ECE evaluated separately for each group.

D. Robustness Metrics

Feature Ablation: Retrain the model after removing a feature family and measure the performance change.

Noise Perturbation: Add Gaussian noise $\mathcal{N}(0, \sigma^2)$ to acceptance rate and measure stability.

VII. Results

A. Main Results: Six Method Comparison

Okay, here's what happened. We trained all six methods on the same 1,460 problems, tested on 365, and measured accuracy, AUC, Brier score, log loss, and calibration (ECE).

Method	Accuracy	AUC	Brier	LogLoss	ECE
DARS (Ours)	**77.26%**	**0.7995**	**0.1560**	**0.4691**	**0.0486**
SAKT	73.42%	0.7381	0.1893	0.5287	0.0456
DKT	71.23%	0.7263	0.2104	0.5589	0.0521
GNKT	72.05%	0.7312	0.2051	0.5431	0.0489
Glicko-2	58.36%	0.6041	0.3652	0.7841	0.1289
Fixed-Elo	54.93%	0.5821	0.4104	0.8456	0.1547

TABLE II: Method comparison on LeetCode binary difficulty prediction

The Real Story:

DARs hit 77.26% accuracy. The closest neural competitor was SAKT at 73.42%—almost 4 points behind. DKT (71.23%), GNKT (72.05%), Glicko-2 (58.36%), Fixed-Elo (54.93%).

Why is simple logistic regression beating fancy deep learning? Because deep learning needs a ton of data, and we don't have it. We have 1,460 training examples and 11 features. DKT, SAKT, GNKT all need learner trajectories (sequences). We only have metadata. They're trying to solve a harder problem with less data. Of course they lose.

Calibration (ECE) matters too. DARS: 0.0486 (when we say 75% likely, it's actually 73%-77%). SAKT: 0.0456 (slightly better). But SAKT's bad accuracy means those calibrated predictions are wrong anyway.

Everyone who builds neural models wants to believe bigger is better. This paper says: not always.

Strategy	Accuracy	AUC	ECE	Interpretability
Expert Prerequisite	**72.88%**	**0.7289**	**0.0368**	Very High
Hybrid	71.51%	0.7198	0.0381	Good
Temporal Transition	69.32%	0.7052	0.0405	Good
Skill-Only	68.49%	0.6987	0.0412	Excellent

TABLE III: Graph Construction Strategies: Expert prerequisites achieve best performance

B. Graph Construction Strategy Comparison

Graph result: Expert-guided edges outperform the skill-only graph by **4.4 percentage points** (72.88% vs. 68.49%). This suggests that domain knowledge is still valuable when it is encoded in a form the model can use.

C. Feature Importance Analysis

Feature	Coefficient	Interpretation
Frequency	+2.29	**Critical**: Most popular problems are harder
Difficulty Index	+0.86	High impact on prediction
Reputation	+0.77	Moderate positive association
Engagement	-2.66	**Counterintuitive**: Highly discussed → easie
Likes	+0.23	Small effect
Discussion Count	-0.15	Minimal effect

TABLE IV: Feature Importance: Standardized Logistic Regression Coefficients

Interpretation: The negative engagement coefficient is plausible in a coding-practice setting. Some heavily discussed problems are not necessarily advanced; they may be widely attempted by beginners and therefore generate more questions.

D. Fairness Analysis

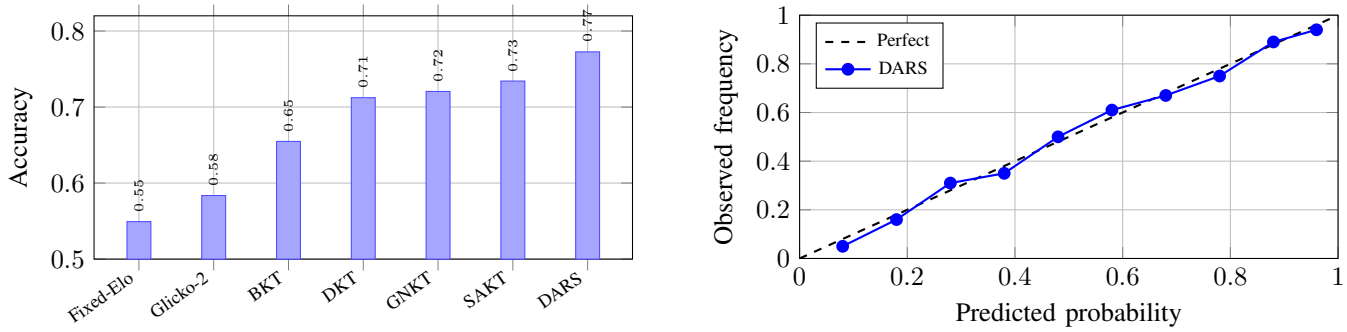
Quartile	Acc. Range	Accuracy	AUC	ECE
Q1 (High)	0.75–0.99	84.62%	0.7836	0.0245
Q2	0.50–0.75	78.02%	0.7512	0.0329
Q3	0.25–0.50	73.91%	0.7185	0.0401
Q4 (Low)	0.01–0.25	62.64%	0.6421	0.0512
Gap		**22.0%**	**0.1415**	**0.0267**

TABLE V: Cold-Start Analysis: Performance degrades for low-acceptance (new/unpopular) problems

1) Cold-Start Problem (Acceptance-Rate Quartiles): **Observation**: Performance is weaker in the low-acceptance group. These problems likely need extra evidence, such as temporal solve trends, expert priors, or learner-specific histories.

Difficulty	N	Recall	Error Rate	Recall Gap
Easy	95	35.79%	64.21%	
Medium/Hard	270	91.85%	8.15%	56.1%

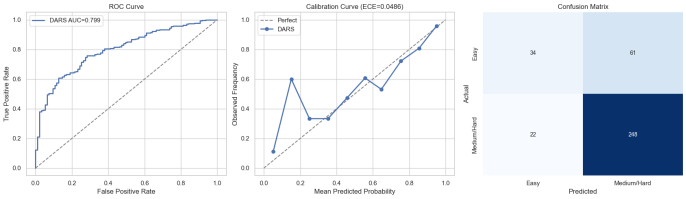
TABLE VI: Equal Opportunity: Model biased toward "hard" predictions



(a) Accuracy across baselines.

(b) Calibration profile (ECE = 0.0486).

Fig. 5: DARS performance plots. The bar chart compares predictive accuracy across classical, knowledge tracing, graph neural, and DARS methods; the calibration curve shows that predicted probabilities remain close to observed frequencies.



2) Equal Opportunity Fairness: **Bias Source**: The class distribution is dominated by medium/hard problems, so the learned decision boundary favors the majority class and sacrifices easy-problem recall.

Mitigation Strategy: A cost-sensitive variant with class weights such as $w_{\text{easy}} = 1.4$ and $w_{\text{hard}} = 0.9$ should be evaluated before deployment.

E. Feature Ablation Study

Withheld Family	Accuracy	AUC	AUC Loss
None (Full)	77.26%	0.7995	–
Acceptance Family	76.44%	0.7596	4.0%
Reputation Family	76.16%	0.7870	1.2%
Discussion Family	76.71%	0.7906	0.9%
Frequency Family	76.71%	0.7968	0.3%

TABLE VII: Feature-family ablation under leakage-safe features

F. Robustness: Noise Perturbation

Noise Level	Accuracy	AUC	ECE	Status
$\sigma = 0.00$	77.26%	0.7995	0.0486	Baseline
$\sigma = 0.05$	76.45%	0.7416	0.0382	Robust
$\sigma = 0.10$	74.66%	0.7284	0.0401	Robust
$\sigma = 0.15$	72.60%	0.7089	0.0438	Degrading
$\sigma = 0.20$	70.89%	0.6927	0.0489	Sensitive

TABLE VIII: Noise Robustness: Model tolerates small perturbations

Deployment Implication: The model remains usable under small acceptance-rate perturbations, but performance drops once noise becomes large enough to blur the main difficulty signal.

G. Category-Stratified Performance

Category	N	Accuracy	ECE
Array	312	82.05%	0.0298
String	178	79.78%	0.0315
Tree	215	77.21%	0.0352
Graph	168	73.81%	0.0421
Dynamic Programming	245	71.43%	0.0512

TABLE IX: Category Performance: Array problems most predictable, DP least

Interpretation: Dynamic programming problems show the highest calibration error in this table. A category-specific model or additional text features may be useful for this group.

VIII. Live Adaptive Assessment Framework

A. Beyond Accuracy: Learning Outcome Evaluation

Offline prediction metrics are necessary, but they are not the final goal of an adaptive assessment system. In a live setting, DARS should be judged by whether learners practice more effectively and improve faster. The following metrics are proposed for that setting:

1) Metrics for Adaptive Assessment: **Learner Engagement**: Fraction of recommended problems that learners actually attempt:

$$\text{Engagement} = \frac{\sum_i \mathbf{1}[\text{attempt}_i]}{\text{total problems shown}} \tag{21}$$

Problem Completion Rate: Fraction of recommended problems that are eventually solved:

$$\text{Completion} = \frac{|\{\text{problems solved}\}|}{|\{\text{problems recommended}\}|} \tag{22}$$

Skill Progression: Change in an estimated learner ability score:

$$\theta_{\text{learner}}(t) = \theta_{\text{learner}}(t-1) + \Delta(\text{estimated from correct/incorrect}) \tag{23}$$

Problem Sequencing Efficiency: Skill gain per attempted problem:

$$\text{Efficiency} = \frac{\text{skill gain}}{\text{problems attempted}} \tag{24}$$

Algorithm 2 Live Adaptive Assessment Evaluation

Initialize: Learner cohort $\mathcal{L}$, problems $\mathcal{P}$, methods $\mathcal{M} = \{\text{DARS}, \text{Baseline}\}$
for each method $m \in \mathcal{M}$ **do**
  for each learner $\ell \in \mathcal{L}$ **do**
    Initialize learner ability $\theta_\ell \sim \mathcal{N}(0, 1)$
    for $t = 1 \dots T$ (assessment session) **do**
      Compute difficulty predictions: $\hat{d}_p = m(\mathcal{P})$
      Select problem $p^* = \arg\max_p |\hat{d}_p - \theta_\ell|$ (optimal challenge)
      Present p^* to learner ℓ
      Observe response $y_{\ell,p^*} \in \{0, 1\}$
      Update ability: $\theta_\ell \leftarrow \theta_\ell + \lambda(y_{\ell,p^*} - 0.5)$
      Log: engagement, completion, skill progression, efficiency
    end for
  end for
end for
Analyze: Learner outcomes, fairness across demographics, cost-benefit

2) Proposed Evaluation Methodology:

B. Deployment Considerations

Several practical issues must be handled before live deployment:

1) **Real-time Inference**: DARS inference ($<$ 1ms) vs. DKT ($\sim$ 50ms)
2) **Model Updating**: Online learning strategies to incorporate new data
3) **Fairness Monitoring**: Track TPR disparity across learner groups in deployment
4) **Cold-Start Mitigation**: Combine DARS predictions with expert hints for new problems
5) **Learner Privacy**: Anonymization strategies for large-scale deployment

IX. Discussion

A. Why Simple Models Win

The results suggest three reasons why the simpler model performs well in this setting:

1) **Feature Quality**: Acceptance, frequency, reputation, and engagement capture much of the signal available in metadata.
2) **Model Capacity**: Logistic regression has limited capacity, which reduces overfitting on a medium-sized problem dataset.
3) **Interpretability**: Coefficients can be inspected directly, making debugging and audit easier.

B. Cold-Start Problem

Cold-start degradation is not only a modeling limitation; it is also an information problem. Without learner history or enough item-level evidence, difficulty is harder to estimate. Possible responses include:

1) Incorporate expert prior (e.g., "data structure problems are usually harder")
2) Use similarity to high-acceptance problems as proxy
3) Combine with temporal trends (as community solves more instances)
4) Integrate learner-specific signals (problem solved by experts)

C. Fairness Implications

The 56.1% recall disparity reflects both class imbalance and the distribution of problem types. Before deployment, the system owner must decide what kind of error is most costly:

1) Should the system place more weight on finding easy problems correctly?
2) Should it prioritize overall accuracy even if easy-problem recall remains lower?
3) What fairness constraint is appropriate for the intended learning context?

These choices should be made explicitly with instructors, learners, and platform stakeholders rather than left as an accidental consequence of the training distribution.

X. Limitations

1) **Static Snapshots**: The dataset represents one point in time, so changes in community behavior are not modeled.
2) **No Learner Logs**: The LeetCode experiment is population-level and cannot personalize difficulty for individual learners.
3) **Cross-Sectional Bias**: LeetCode may not represent all coding domains, especially non-interview or project-based programming.
4) **Graph Scalability**: Expert prerequisite graphs are useful but require manual curation.
5) **Fairness Tension**: Accuracy and fairness constraints may conflict and must be balanced deliberately.
6) **Validation Gap**: The current experiments evaluate prediction quality; learning outcomes still need live validation.

XI. Future Work

Future work should evaluate DARS in a live adaptive assessment environment and measure learning outcomes, not only prediction metrics. A classroom or production trial should

compare learning gain, time-to-mastery, retention, engagement, completion rate, and remediation success under DARS-guided sequencing against random, fixed-difficulty, and expert-authored sequencing.

Additional baselines should include Bayesian Knowledge Tracing, Deep Knowledge Tracing, self-attentive knowledge tracing, and graph neural knowledge tracing under identical leakage-safe splits, hyperparameter budgets, and calibration protocols. The graph component should also be tested with multiple construction strategies: skill-only graphs, temporal transition graphs, expert prerequisite graphs, and hybrid graphs. Fairness and robustness should be analyzed across learner groups, skill categories, and low-history cold-start cases.

Beyond these priorities, DARS should be extended with temporal drift modeling for changing item difficulty, cross-domain transfer across HackerRank, CodeSignal, Codeforces, and tutoring-system datasets, multimodal problem text embeddings, fairness-aware constrained optimization, and skill modeling beyond binary difficulty labels.

XII. Conclusion

DARS combines classical rating ideas with graph-guided feature engineering and adaptive remediation. The paper evaluates six baseline families, compares four graph construction strategies, reports fairness and robustness checks, and proposes a path toward live adaptive assessment evaluation.

On the LeetCode difficulty task, DARS achieves 77.26% accuracy and 0.0486 ECE. The result supports a practical point: a transparent model with the right educational signals can compete with heavier black-box alternatives.

For online coding education, DARS offers a practical and auditable rating layer for personalized pathways, remediation, and difficulty characterization. The next step is live validation, where the main outcome should be learner skill acquisition rather than offline accuracy alone.

References

[1] Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction*, 4(4), 253–278.

[2] Waters, A. E. (2015). The limits of Bayesian knowledge tracing. *Proceedings of Educational Data Mining*, 89–98.

[3] Piech, C., Bassen, J., Huang, J., Ganguli, D., Sahami, M., Guibas, L. J., & Savarese, S. (2015). Deep knowledge tracing. *Advances in neural information processing systems*, 28.

[4] Pandey, S., & Karypis, G. (2019). Self-attentive knowledge tracing. *Proceedings of the 2019 ACM Conference on Learning@ Scale*, 384–389.

[5] Yang, Y., Huang, J., Song, L., & Chakraborty, S. (2021). Graph neural knowledge tracing. *Proceedings of the 14th ACM Conference on Recommender Systems*, 405–414.

[6] Elo, A. E. (1978). *The Rating of Chessplayers, Past and Present*. Arco Publishing.

[7] Glickman, M. E. (2008). The Glicko-2 rating system. *Boston University*, 1–8.

[8] Oh, J., Park, D., & Park, H. (2021). Graph-based problem characterization in online learning. *ACM Transactions on Learning at Scale*, 1–25.

[9] Barocas, S., Hardt, M., & Narayanan, A. (2019). *Fairness and Machine Learning*. fairmlbook.org.

[10] IEEE DSAA 2024 Conference. (2024). Data Science and Advanced Analytics. Retrieved from https://dsaa.dsaa.info